

ELO320 - Estructuras de Datos y Algoritmos

Ayudantía 4

Departamento de Electrónica

Ingeniería Civil Telemática

Gabriel Zapata, Byron Agurto, Lucas Siclari.



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

LIDER EN
INGENIERIA, CIENCIA
Y TECNOLOGIA

1 Structs

2 Parámetros de Ejecución

3 Lectura de Archivos

4 Ejercicios propuestos



El Struct

Las Estructuras o *Structs* son una manera para organizar distintas variables en una composición personalizada.

```
4 struct persona{
5     int edad;
6     char sexo;
7     char nombre[30];
8     char apellido[30];
9 };

11 ∨ int main(){
12     struct persona p1;
13     strcpy(p1.nombre, "pedrito");
14     p1.edad = 25;
15     printf("%s tiene %d anos\n", p1.nombre, p1.edad);
16     return 0;
17 }
```



El Struct

Las Structs que ya se hayan definido pueden ser declaradas como atributos en tu Struct actual.

¿Pueden identificar las diferentes formas de implementación?

```
3  #include "ola.c"
4  typedef struct empresa{
5      struct persona personal;
6      int personal_total;
7  } empresa;

9  int main(){
10     struct persona p1;
11     strcpy(p1.nombre, "pedrito");
12     p1.edad = 25;
13     empresa e1;
14     e1.personal = p1;
15     char frase[] = "%s tiene %d anos y trabaja con nosotros\n";
16     printf(frase, e1.personal.nombre, e1.personal.edad);
17     return 0;
18 }
```



Parámetros por CMD

```
argc y **argv
```

Pasar parámetros por consola inserta strings en el "vector de argumentos" y que nuestros programas pueden utilizar según las instrucciones que coloquemos.

```
4  ∨ int main(int argc, char **argv){
5  ∨     if(argc==1){
6      |     printf("¡Mete más argumentos al ejecutar!");
7      |     return -1;
8      | }
9
10     for(int i=0;i<argc;i++)
11         printf("Argumento %i: %s \n",i, argv[i]);
12
13     return 0;
14 }
```

```
PS C:\Users\Pablo\Desktop\dev\C> .\MAIN.exe ola a ae ei oe
Argumento 0: C:\Users\Pablo\Desktop\dev\C\MAIN.exe
Argumento 1: ola
Argumento 2: a
Argumento 3: ae
Argumento 4: ei
Argumento 5: oe
```



Lectura de Archivos

Como vimos antes, uno pasa strings al vector de argumentos. Entonces, para leer un archivo, debemos pasar su nombre como tal y que el programa lo abra y lea.

```
12     file = fopen(argv[1], "r");
13     if (file == NULL) {
14         perror("Error al abrir el archivo");
15         return 1;
16     }
```

```
PS C:\Users\Pablo\Desktop\dev\C> .\MAIN.exe archivito.csv
```

```
Linea 1: 45,88,19
```

```
Linea 2: 74,31,92
```

```
Linea 3: 23,55,66
```

```
PS C:\Users\Pablo\Desktop\dev\C> .\MAIN.exe archivito.txt
```

```
Linea 1: hola
```

```
Linea 2: soy archivito
```

```
Linea 3: pero en txt
```



Lectura de Archivos pasados por CMD

```
1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      FILE *file;           // Puntero FILE que recorre el archivo desde un stream
5      char line[256];       // Buffer para leer cada línea del archivo
6      int row = 0;          // Contador de filas
7      if (argv[1] == NULL) {
8          perror("Ningun archivo ingresado");
9          return 1;
10     }
11
12     file = fopen(argv[1], "r");
13     if (file == NULL) {
14         perror("Error al abrir el archivo");
15         return 1;
16     }
17
18     // Leer el archivo línea por línea
19     while (fgets(line, sizeof(line), file)) {
20         printf("Linea %d: %s", row + 1, line);
21         row++;
22     }
23     fclose(file); // Cerrar el archivo
24
25     return 0;
26 }
```



Ejercicios 1

Vehículo Creator

Cree un programa que incluya el struct "carroza" y el struct "motor". Carroza tendrá atributos para la cantidad de ruedas, un string para el color y un struct motor. El struct motor contendrá los atributos de la Potencia y Nivel de Sonorización (mientras más alto más la gente te odiará por las calles).

Intente visualizar qué implementar primero, ¿el motor o la carroza? ¿Qué va dentro de qué?

Su propósito final es que el programa reciba parámetros por consola y el vehículo de tus sueños sea objetivo de los delincuentes en villa Autolandia.

Ayudante de Fibonacci

Cree un programa en C que abra un archivo de texto llamado "fibonacci.txt" para escritura, y luego escriba en él los primeros 10 números de la serie de fibonacci (puede utilizar la función de la ayudantía 2) separados por saltos de línea . Después de escribir los datos, cierra el archivo.



Catastro de Reclusos

Cree el struct "Recluso" con los atributos nombre y los años que estará privado de libertad. El programa debe tomar tres argumentos por comando: "reclusos.txt", el nombre y los años. El programa debe abrir el archivo especificado, escribir los datos del recluso y cerrarse. Al ejecutar de nuevo el programa, el archivo debe seguir insertando reclusos **sin** sobrescribir los anteriores.

Juez de Libertad

Escriba un programa que lea el archivo del ejercicio anterior. Este programa recibe tres parámetros por consola: "reclusos.txt", el nombre de un recluso y un valor numérico. El valor numérico indica si el recluso es: Liberado (se borra del archivo), Ejecutado (su tiempo de recluso es reescrito a 0) o es Contado (indica la cantidad de veces que este nombre se repite en el archivo).



¡¡¡Felicitaciones!!!

